

SKILL WEEK- 5

1. SQL

Query 1 x

Limit to 1000 rows

```
1 • CREATE DATABASE university;
2 • USE university;
3 • CREATE TABLE students (
4     id INT PRIMARY KEY,
5     name VARCHAR(100),
6     department VARCHAR(100)
7 );
8 • SELECT * FROM students;
9 • INSERT INTO students (id, name, department)
10 VALUES
11 (101, 'Ravi', 'CSE'),
12 (102, 'Priya', 'ECE'),
13 (103, 'Arun', 'CSE'),
14 (104, 'Sneha', 'IT'),
15 (105, 'Rahul', 'EEE');
16 • SELECT * FROM students;
17
```

12 (102, 'Priya', 'ECE'),
13 (103, 'Arun', 'CSE'),
14 (104, 'Sneha', 'IT'),
15 (105, 'Rahul', 'EEE');
16 • SELECT * FROM students;
17

Result Grid | Filter Rows:

	id	name	department
▶	0	string	string
	101	Ravi	CSE
	102	Priya	ECE
	103	Arun	CSE
	104	Sneha	IT
	105	Rahul	EEE
	106	Harshika	CS-IT
*	NULL	NULL	NULL

students 2 x

1. Code

```
api.py > get_students
1  from fastapi import FastAPI
2  from sqlalchemy import create_engine, text
3  from pydantic import BaseModel
4  app = FastAPI()
5  DATABASE_URL = "mysql+pymysql://root:Harshu%402208@localhost/university"
6  engine = create_engine(DATABASE_URL)
7  @app.get("/students")
8  def get_students():
9      with engine.connect() as connection:
10         result = connection.execute(
11             text("SELECT * FROM students")
12         )
13         students = result.mappings().all()
14         return students
15 class Student(BaseModel):
16     id: int
17     name: str
18     department: str
19 @app.post("/students")
20 def create_student(student: Student):
21     with engine.begin() as connection:
22         connection.execute(
23             text("""
24                 INSERT INTO students (id, name, department)
25                 VALUES (:id, :name, :department)
26             """),
27             {
28                 "id": student.id,
29                 "name": student.name,
30                 "department": student.department
31             }
32         )
33     return {
34         "message": "Student created",
35         "student": student
36     }
```

2. FAST API

GET

/students

Get Students

⬆

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/students' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/students
```

Server response

CodeDetails

200

Response body

```
{
  "id": 102,
  "name": "Priya",
  "department": "ECE"
},
{
  "id": 103,
  "name": "Arun",
  "department": "CSE"
},
{
  "id": 104,
  "name": "Sneha",
  "department": "IT"
},
{
  "id": 105,
  "name": "Rahul",
  "department": "EEE"
},
{
  "id": 106,
  "name": "Harshika",
  "department": "CS-IT"
}
]
```

Download

Response headers

```
content-length: 320
content-type: application/json
```

POST

/students

Create Student

⬆

Parameters

Cancel

Reset

No parameters

Request body

required

application/json

Edit Value | Schema

```
{
  "id": 107,
  "name": "Siri",
  "department": "ECE"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
'http://127.0.0.1:8000/students' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "id": 107,
  "name": "Siri",
  "department": "ECE"
}'
```

Request URL

```
http://127.0.0.1:8000/students
```

Server response

CodeDetails

200

Response body

```
{
  "message": "Student created",
  "student": {
    "id": 10,
    "name": "Sripal",
    "department": "ECE"
  }
}
```

Download

Response headers

```
content-length: 83
content-type: application/json
date: Wed, 16 Sep 2026 09:28:39 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header. Example Value Schema "string"	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": [0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	No links